Node.js Backend: недели 1–2

Основа: современный JS, асинхронность, устройство Node, TypeScript для сервера, тесты, streams

Нагрузка: ~3 часа в будни (3 × 1 ч) + 8–10 часов на выходных. Документ рассчитан на разработчика, уже знакомого с TypeScript/Angular.

Содержание

1. Как пользоваться документом
2. Неделя 1, день 1 — окружение и проект
3. Неделя 1, день 2 — современный JS для бэкенда
4. Неделя 1, день 3 — асинхронность
5. Неделя 1, суббота — как устроен Node: event loop, модули, process, fs
6. Неделя 1, воскресенье — задача 1 (базовая версия)
7. Неделя 2, дни 1–2 — TypeScript на сервере
8. Неделя 2, день 3 — тесты на node:test
9. Неделя 2, суббота — задача 1 на TS + тесты + «Уровень+»
10. Неделя 2, воскресенье — streams и задача 2
11. ТЗ задачи 1: CLI-менеджер заметок
12. ТЗ задачи 2: анализатор логов
13. Ответы на упражнения и источники

1. Как пользоваться документом

- **Будни (1 ч):** прочитать раздел дня и сделать упражнения в конце раздела. Упражнения короткие (15–30 минут) — их задача закрепить, а не нагрузить.
- **Выходные:** теория 1.5–2 ч, остальное — задача. Код важнее чтения.
- Все упражнения кладите в репозиторий `node-learning/`: папки `exercises/week1`, `exercises/week2`, `notes-cli/`, `log-analyzer/`.
- Если что-то из раздела вы уже знаете по Angular — пробегите глазами и сразу переходите к упражнениям. Если упражнение решается за 2 минуты — тема закрыта.
- В конце каждой недели есть блок «Проверь себя». Если на какой-то вопрос не можете ответить вслух — вернитесь к разделу.

> **Главное отличие от фронтенда**
> В браузере вы живёте внутри страницы: DOM, события, fetch. На сервере вы живёте внутри **процесса операционной системы**: файлы, переменные окружения, аргументы, сигналы, коды выхода, сеть, сотни одновременных клиентов. Большая часть первых двух недель — именно про это.

Неделя 1

День 1 Окружение и проект

Node у вас уже есть, но для бэкенда важно уметь **переключать версии**: разные проекты требуют разный Node.

Менеджер версий

```
# nvm
nvm ls                        # какие версии установлены
nvm install --lts             # поставить актуальную LTS
nvm use --lts
nvm alias default lts/*       # LTS по умолчанию в новых терминалах
node -v
```

В корне проекта создайте файл `.nvmrc` с номером версии (например, `node -v > .nvmrc`). Тогда в папке проекта достаточно набрать `nvm use` без аргументов, а CI и коллеги будут на той же версии.

Angular и бэкенд на разных версиях Node
Если рабочий Angular-проект требует другую версию Node — ничего страшного: у каждого проекта свой `.nvmrc`, переключение — `nvm use` в папке проекта.

Почему именно LTS
Чётные версии Node (22, 24, 26…) получают статус LTS — долгую поддержку. Нечётные живут ~6 месяцев и нужны для пробы новых фич. В продакшене — только LTS.

Структура репозитория

```
node-learning/
  exercises/
    week1/
    week2/
  notes-cli/          ← задача 1
  log-analyzer/       ← задача 2
  .gitignore
  .nvmrc
  README.md
```

`.gitignore` минимум: `node_modules/`, `.env`, `*.log`, `dist/`, `coverage/`.

package.json — что важно на бэкенде

```
{
  "name": "notes-cli",
  "version": "0.1.0",
  "type": "module",               // ESM: import/export вместо require
  "engines": { "node": ">=22" },  // какая версия Node нужна
  "scripts": {
    "start": "node src/cli.ts",
    "dev": "node --watch src/cli.ts",
    "test": "node --test test/*.test.ts",
    "typecheck": "tsc --noEmit"
  },
  "dependencies": {},             // нужно в рантайме
  "devDependencies": {}           // только для разработки (typescript, @types/node)
}
```

- `npm ci` vs `npm install`: `ci` ставит строго по `package-lock.json` и падает при расхождении — используется в CI и Docker.
- `package-lock.json` **всегда коммитится**.
- `npx <tool>` — запустить бинарник из `node_modules/.bin` или скачать временно.
- `node --watch file` — встроенный перезапуск при изменении файлов (nodemon больше не нужен).

День 2 Современный JS для бэкенда

Вы это частично знаете — фокус на том, что на сервере используется постоянно.

1. Деструктуризация, rest, spread

```
const user = { id: 1, name: 'Ann', role: 'admin', password: 'hash...' };

const { name: userName, age = 18 } = user;   // переименование + значение по умолчанию

// rest — убрать поле. Классика бэкенда: не отдать пароль клиенту
const { password, ...publicUser } = user;

// spread — иммутабельное обновление
const updated = { ...user, name: 'Bob', updatedAt: new Date().toISOString() };

// "options object" в параметрах — основной стиль API в Node-библиотеках
function createNote({ text, tags = [], done = false }) { /* ... */ }
```

Поверхностная копия

`{...obj}` копирует только верхний уровень — вложенные объекты общие. Глубокая копия: `structuredClone(obj)` (встроено в Node).

2. `?.` и `??` — разница с `||` критична

```
const city  = body?.address?.city;   // undefined вместо TypeError
const limit = query.limit ?? 20;      // 20 только если null/undefined
const bad   = query.limit || 20;      // 0 тоже станет 20 — баг!
const done  = input.done ?? false;    // false сохранится, а не перезапишется
```

На сервере `0`, `''`, `false` — нормальные значения (`page=0`, `done=false`, пустой комментарий). Используйте `??` по умолчанию.

3. Методы массивов

Метод	Возвращает	Когда
`map`	новый массив той же длины	преобразовать каждый элемент (DB-строка → DTO)
`filter`	новый массив	отобрать
`find` / `findIndex`	элемент или `undefined` / индекс или `-1`	поиск по id
`findLast` / `findLastIndex`	то же, с конца	последняя запись по условию
`some` / `every`	`boolean`	проверки прав, валидация
`reduce`	что угодно	суммы, словари, группировки
`flatMap`	плоский массив	у каждого поста массив тегов → все теги
`toSorted`, `toReversed`, `toSpliced`, `with`	**новый** массив	иммутабельные версии `sort` / `reverse` / `splice` / `arr[i]=x`

```
const orders = [
  { id: 1, userId: 10, total: 50, status: 'paid' },
  { id: 2, userId: 11, total: 30, status: 'new'  },
  { id: 3, userId: 10, total: 20, status: 'paid' },
];

const revenue = orders
  .filter(o => o.status === 'paid')
  .reduce((sum, o) => sum + o.total, 0);               // 70

const totalByUser = orders.reduce((acc, o) => {
  acc[o.userId] = (acc[o.userId] ?? 0) + o.total;
  return acc;
}, {});                                                 // { 10: 70, 11: 30 }

const byStatus = Object.groupBy(orders, o => o.status); // { paid: [...], new: [...] }
const top = orders.toSorted((a, b) => b.total - a.total)[0];
```

Ловушка: `sort()` мутирует и сортирует как строки
`[10, 9, 1].sort()` → `[1, 10, 9]`. Всегда передавайте компаратор: `(a, b) => a - b`. Для строк с кириллицей: `(a, b) => a.localeCompare(b, 'ru')`.

4. Map и Set — когда объекта мало

```
// Set — уникальные значения, проверка за O(1)
const uniqueIps = new Set(['1.1.1.1', '2.2.2.2', '1.1.1.1']);   // size: 2
uniqueIps.has('1.1.1.1');                                        // true
const arr = [...uniqueIps];

// Map — словарь с любыми ключами, сохраняет порядок вставки, быстрый size
const counter = new Map();
for (const ip of ips) counter.set(ip, (counter.get(ip) ?? 0) + 1);

const top5 = [...counter.entries()]
  .sort((a, b) => b[1] - a[1])
  .slice(0, 5);

Object.fromEntries(counter);   // Map → обычный объект (для JSON)
new Map(Object.entries(obj));  // объект → Map
```

Правило: данные-сущности (user, note) — объекты; динамические словари и счётчики (по IP, по id) — `Map`; «множество уникальных» — `Set`. В задаче 2 Map будет основным инструментом.

5. Даты (минимум)

```
new Date().toISOString();                    // '2026-11-11T14:30:00.000Z' — храним и
передаём так
Date.now();                                  // миллисекунды, удобно для замеров
new Date(iso).toLocaleString('ru-RU');       // показываем пользователю так
```

Правило бэкенда: **хранить в UTC (ISO-строка или timestamp), форматировать только при выводе**.

Упражнения дня 2 (файл `exercises/week1/day2.js`)

1. `groupBy(items, key)` — своя реализация через `reduce` (без `Object.groupBy`).
2. `findDuplicates(arr)` → массив значений, встречающихся больше одного раза. Через `Set` / `Map`, за один проход.
3. `chunk(arr, size)` → `chunk([1,2,3,4,5], 2) = [[1,2],[3,4],[5]]` (пригодится для батч-вставки в БД).
4. `countBy(arr, fn)` → `Map` «ключ → количество».
5. `uniqueBy(arr, key)` → убрать дубли по полю, оставить первое вхождение.
6. `pick(obj, keys)` и `omit(obj, keys)` — без мутации исходного объекта.

Проверяйте результат через `console.log` или сразу через `assert`: `import assert from 'node:assert/strict'`.

День 3 Асинхронность

Вы привыкли к RxJS — в Node основной инструмент **Promise + async/await**. Observable на бэкенде почти не встречается; ближайший аналог «потока значений» — streams и async-итераторы (неделя 2).

1. Promise: три состояния

`pending` → `fulfilled` (с значением) или `rejected` (с ошибкой). Переход происходит один раз и навсегда.

```
const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

// Встроенная версия, писать самому не обязательно:
import { setTimeout as delay } from 'node:timers/promises';
await delay(500);
```

2. async/await и ошибки

```
async function loadUser(id) {
  try {
    const raw = await readFile(`users/${id}.json`, 'utf8');
    return JSON.parse(raw);
  } catch (err) {
    if (err.code === 'ENOENT') return null;   // ожидаемая ситуация — обрабатываем
    throw err;                                 // неожиданная — пробрасываем дальше
  }
}
```

Важный принцип: ловите только те ошибки, которые знаете, как обработать. Остальные пробрасывайте наверх. «Проглотить» ошибку в `catch {}` — один из худших багов на бэкенде.

3. Последовательно vs параллельно

```
// Последовательно: 3 × 1c = 3c
const a = await fetchA();
const b = await fetchB();
const c = await fetchC();

// Параллельно: ~1c
const [a, b, c] = await Promise.all([fetchA(), fetchB(), fetchC()]);
```

Комбинатор	Результат	Когда использовать
`Promise.all`	массив значений; падает на первой ошибке	нужны все результаты
`Promise.allSettled`	массив `{status, value | reason}`, никогда не падает	рассылка уведомлений: часть может упасть
`Promise.race`	первый завершившийся (успех или ошибка)	таймауты
`Promise.any`	первый успешный	запрос к нескольким зеркалам

4. Частые ошибки

```
// 1) async внутри forEach — forEach НЕ ждёт промисы
items.forEach(async (item) => { await save(item); });
console.log('готово');               // выведется ДО сохранения!

// Правильно — последовательно:
for (const item of items) await save(item);
// или параллельно:
await Promise.all(items.map((item) => save(item)));

// 2) забытый await — функция вернёт Promise, а не данные
const user = getUser(1);             // Promise { <pending> }
if (user.isAdmin) { ... }            // всегда undefined

// 3) Promise.all на 10 000 элементов — 10 000 одновременных запросов к БД.
//    Решение: батчи (ваша функция chunk из дня 2!)
for (const batch of chunk(items, 50)) {
  await Promise.all(batch.map(save));
}
```

Unhandled rejection = падение процесса

В современном Node необработанный rejected-промис **завершает процесс**. В браузере вы бы увидели ошибку в консоли, на сервере — упавший сервис. Каждый промис должен где-то заканчиваться `await` внутри `try/catch` или `.catch()`.

5. Отмена операций: AbortController

```
const controller = new AbortController();
setTimeout(() => controller.abort(), 2000);

try {
  const res = await fetch('https://example.com/slow', { signal: controller.signal });
} catch (err) {
  if (err.name === 'AbortError') console.log('таймаут');
}

// Короче — встроенный таймаут:
await fetch(url, { signal: AbortSignal.timeout(2000) });
```

В Node `fetch` встроен глобально, как в браузере. `signal` принимают многие API Node (`readFile`, `setTimeout` из `timers/promises` и др.).

Упражнения дня 3 (`exercises/week1/day3.js`)

1. Напишите `fakeRequest(name, ms)` , которая через `ms` миллисекунд возвращает `name` . Запустите три запроса (300, 500, 700 мс) последовательно и параллельно, замерьте время через `performance.now()` .
2. `withTimeout(promise, ms)` через `Promise.race` : если промис не успел — reject с `Error('Timeout')` .
3. `retry(fn, attempts, delayMs)` : вызывает async-функцию, при ошибке повторяет до `attempts` раз с паузой. Протестируйте функцией, которая падает первые 2 раза.
4. Воспроизведите ошибку с `forEach` + async, затем исправьте двумя способами.
5. Через `Promise.allSettled` «отправьте» 5 уведомлений, 2 из которых падают; выведите «доставлено: 3, ошибок: 2».
6. *Со звёздочкой: `mapWithConcurrency(items, limit, fn)` — обработать массив, но не более `limit` операций одновременно.

Суббота Как устроен Node

Теория 1.5–2 ч. Обязательно к просмотру: доклад Philip Roberts «What the heck is the event loop anyway?» (JSConf EU, YouTube, ~25 мин) и статья «The Node.js Event Loop» на nodejs.org/learn.

1. Из чего состоит Node

- **V8** — движок JavaScript (тот же, что в Chrome). Выполняет ваш код в **одном потоке**.
- **libuv** — C-библиотека: event loop, асинхронный ввод-вывод, **пул потоков** (по умолчанию 4) для операций, которые ОС не умеет делать асинхронно (файловая система, DNS, криптография, сжатие).
- **Встроенные модули** (`node:fs`, `node:http`, ...) — JS-обёртки над libuv и системными вызовами.

Ключевая идея

Ваш JS-код — однопоточный. Но ожидание (сеть, диск, таймеры) происходит **вне** этого потока. Пока один запрос ждёт ответ от БД, поток свободен и обрабатывает другие. Поэтому один процесс Node держит тысячи соединений.
Обратная сторона: **тяжёлое вычисление в JS блокирует всех**. Цикл на 2 секунды = 2 секунды сервер не отвечает никому.

```
// Плохо в обработчике запроса:
const data = fs.readFileSync('big.json');   // блокирует поток
const hash = crypto.pbkdf2Sync(...);        // блокирует поток
JSON.parse(hugeString);                     // блокирует (неизбежно, но помнить)
while (Date.now() < end) {}                 // блокирует

// *Sync-функции нормальны только при старте приложения или в CLI-скриптах.
```

2. Фазы event loop

Цикл крутится, пока есть что делать (таймеры, открытые сокеты, ожидающие операции). Каждая итерация проходит фазы:

Фаза	Что выполняется
1. timers	колбэки `setTimeout` / `setInterval`, время которых наступило
2. pending callbacks	отложенные системные колбэки (некоторые ошибки TCP)
3. poll	ожидание и обработка событий I/O: данные из сокета, завершение чтения файла
4. check	колбэки `setImmediate`
5. close callbacks	события закрытия (`socket.on('close')`)

Между любыми колбэками опустошаются очереди микрозадач:

1. очередь `process.nextTick` (специфична для Node);
2. очередь промисов (`.then`, `await`, `queueMicrotask`).

Нюанс ESM

В CommonJS-модуле `nextTick` всегда выполняется раньше промисов. Но в ESM-модуле (`"type": "module"`) сам модуль исполняется внутри промиса, поэтому на верхнем уровне файла промисы срабатывают **раньше** `nextTick` . Мы проверили это на Node 22 — см. ответы в конце.

Упражнение: угадай порядок (сначала на бумаге, потом запустить)

```
// order.mjs
console.log('1');
setTimeout(() => console.log('2'), 0);
setImmediate(() => console.log('3'));
Promise.resolve().then(() => console.log('4'));
process.nextTick(() => console.log('5'));
console.log('6');
```

Запустите как `order.mjs` и как `order.cjs` (тот же код). Затем:

```
// io.mjs
import { readFile } from 'node:fs';
readFile(import.meta.filename, () => {
  setTimeout(() => console.log('timeout'), 0);
  setImmediate(() => console.log('immediate'));
});
```

Запустите `io.mjs` 10 раз и `order.mjs` 10 раз. Где порядок timeout/immediate стабилен, а где — нет? Почему? (Ответ — в конце документа.)

3. Модули: ESM vs CommonJS

	ESM (используем)	**CommonJS (легаси, но встречается)**
Синтаксис	`import x from './x.js'` `export function f() {}`	`const x = require('./x')` `module.exports = { f }`
Включается	`"type": "module"` или `.mjs`	по умолчанию или `.cjs`
Расширение в импорте	**обязательно** (`./x.js` , `./x.ts`)	можно опустить
Top-level await	да	нет
Путь к текущему файлу	`import.meta.filename` , `import.meta.dirname`	`__filename` , `__dirname`

Встроенные модули импортируйте с префиксом `node:` — `import { readFile } from 'node:fs/promises'` . Так сразу видно, что это не пакет из npm.

4. Объект process

```
process.argv          // ['/path/node', '/path/cli.js', 'add', 'Купить молоко']
process.argv.slice(2) // ['add', 'Купить молоко'] — аргументы пользователя
process.env.NOTES_FILE        // переменные окружения (всегда строки или undefined!)
process.cwd()                 // папка, из которой запустили (не папка скрипта!)
process.exitCode = 1          // код выхода; процесс завершится сам, когда всё доделает
process.exit(1)               // немедленный выход — оборвёт незаписанный вывод/файлы
process.on('SIGINT', ...)     // Ctrl+C — пригодится для graceful shutdown
process.memoryUsage().rss     // сколько памяти занимает процесс (байты)
```

- **Код выхода** — способ программы сказать «получилось / нет» другим программам: shell-скриптам, CI, Docker. `0` = успех, любое другое = ошибка. Проверка: `node cli.js; echo $?`.
- `stdout` (`console.log`) — результат работы; `stderr` (`console.error`) — ошибки и диагностика. Их можно перенаправить раздельно: `node cli.js list > out.txt 2> err.txt`.
- Предпочитайте `process.exitCode = 1` вместо `process.exit(1)`.

Разбор аргументов без библиотек — встроенный `util.parseArgs`:

```
import { parseArgs } from 'node:util';

const { values, positionals } = parseArgs({
  args: process.argv.slice(2),
  allowPositionals: true,
  options: {
    tag:  { type: 'string', multiple: true },
    json: { type: 'boolean', default: false },
  },
});
// notes add "Купить" --tag shop --tag home --json
// positionals: ['add', 'Купить'], values: { tag: ['shop','home'], json: true }
```

5. Файловая система, path, os

```
import { readFile, writeFile, mkdir, rename, rm, readdir, stat } from 'node:fs/promises';
import path from 'node:path';
import os from 'node:os';

const file = path.join(os.homedir(), '.notes', 'notes.json'); // правильные разделители на
любой ОС
path.dirname(file);    // .../.notes
path.basename(file);   // notes.json
path.extname(file);    // .json
path.resolve('data');  // абсолютный путь относительно process.cwd()

await mkdir(path.dirname(file), { recursive: true }); // создать папки, не падать если есть
const text = await readFile(file, 'utf8');            // без 'utf8' вернётся Buffer
await writeFile(file, JSON.stringify(data, null, 2));
await rename(tmpFile, file);                          // атомарная замена файла
```

Коды ошибок fs — проверяйте `err.code`, а не текст сообщения:

Код	Значение
`ENOENT`	файла/папки не существует
`EACCES` / `EPERM`	нет прав
`EEXIST`	уже существует
`EISDIR`	ожидался файл, а это папка

Путь относительно чего?

`readFile('data.json')` ищет файл относительно `process.cwd()` — папки, **откуда запустили** команду, а не где лежит скрипт. Для файлов рядом со скриптом: `path.join(import.meta.dirname, 'data.json')`.

Упражнения субботы (до задачи 1, ~40 мин)

1. `args.js`: выведите все аргументы и все переменные окружения, начинающиеся с `APP_`. Запустите: `APP_MODE=dev node args.js a b --flag`.
2. `block.js`: `setInterval` печатает «tick» каждые 100 мс; через 1 с запустите синхронный цикл на 2 с. Посмотрите, что произойдёт с тиками.
3. `ls.js <папка>`: выведите содержимое папки — имя, «file/dir», размер. При несуществующей папке — понятное сообщение и код выхода 1.

Затем — **начинайте задачу 1** (ТЗ в разделе 11).

Воскресенье Задача 1: базовая версия

- Базовые команды задачи 1 на чистом JS (можно сразу на TS, если комфортно — Node 22.18+ запускает `.ts` напрямую).
- Вся таблица обработки ошибок из ТЗ.
- README, коммит, push на GitHub.

Проверь себя: неделя 1

1. Почему Node однопоточный, но может обслуживать тысячи соединений одновременно?
2. Чем `readFileSync` отличается от `await readFile` с точки зрения сервера? Когда Sync допустим?
3. Что выведет `[10, 9, 1].sort()` и почему?
4. Когда `??` и `||` дают разный результат? Приведите бэкенд-пример.
5. Почему `async`-колбэк в `forEach` — ошибка?
6. Чем отличаются `Promise.all` и `Promise.allSettled`?
7. Что случится с процессом при необработанном rejected-промисе?
8. Назовите фазы event loop. Где выполняются микрозадачи?
9. Что такое `process.argv[2]`? Чем `process.exitCode = 1` лучше `process.exit(1)`?
10. Что значит `ENOENT`? Почему проверять надо `err.code`, а не `err.message`?
11. Относительно какой папки работает `readFile('x.json')`?

Неделя 2

Дни 1–2 TypeScript на сервере

Синтаксис TS вам знаком по Angular. Здесь — то, что отличается на бэкенде, и темы, которые в Angular-проектах часто проходят мимо.

1. Запуск TS в Node без сборки

Node 22.18+ умеет **type stripping**: `node app.ts` просто вырезает типы и выполняет код. Проверки типов при этом нет — её делает `tsc --noEmit` отдельно (в редакторе и CI).

Из-за «вырезания» нельзя использовать синтаксис TS, который генерирует JS-код:

- `enum` → используйте union-типы или `as const`-объекты;
- `namespace`;
- parameter properties в конструкторе (`constructor(private repo: Repo)`) — то, к чему вы привыкли в Angular DI!

```
// Вместо enum:
const Status = { Todo: 'todo', Done: 'done' } as const;
type Status = (typeof Status)[keyof typeof Status];   // 'todo' | 'done'

// Импорт — с расширением .ts, типы — через import type
import { addNote } from './notes.ts';
import type { Note } from './types.ts';
```

Если версия Node старее или нужен этот синтаксис — `npx tsx app.ts`.

2. tsconfig.json для Node-проекта

```
{
  "compilerOptions": {
    "target": "es2023",
    "module": "nodenext",
    "moduleResolution": "nodenext",
    "strict": true,
    "noUncheckedIndexedAccess": true,   // arr[i] имеет тип T | undefined — ловит реальные
баги
    "noEmit": true,                     // только проверка, запускает сам Node
    "allowImportingTsExtensions": true, // разрешить import './x.ts'
    "verbatimModuleSyntax": true,       // требует import type для типов
    "erasableSyntaxOnly": true,         // запрещает enum/namespace — то, что Node не
запустит
    "skipLibCheck": true,
    "types": ["node"]
  },
  "include": ["src", "test"]
}
```

```
npm i -D typescript @types/node
```

3. `unknown` вместо `any`: всё, что приходит снаружи

На бэкенде большая часть данных приходит **извне**: файл, тело запроса, переменные окружения, ответ чужого API. TypeScript не знает, что там на самом деле. `JSON.parse`

возвращает `any` — это дыра в типизации.

```
const data: unknown = JSON.parse(raw);   // честно: «не знаю, что это»
data.notes;                              // ошибка компиляции — и правильно

// Type guard — функция, которая проверяет и «сужает» тип
function isNote(x: unknown): x is Note {
  return (
    typeof x === 'object' && x !== null &&
    'id' in x && typeof x.id === 'number' &&
    'text' in x && typeof x.text === 'string' &&
    'done' in x && typeof x.done === 'boolean'
  );
}

if (Array.isArray(data) && data.every(isNote)) {
  // здесь data: Note[]
}
```

Писать такие проверки руками утомительно — на 4-й неделе их заменит библиотека **Zod**. Но один раз сделать вручную полезно, чтобы понимать, что она делает.

4. Narrowing и discriminated unions

```
type Command =
  | { kind: 'add'; text: string; tags: string[] }
  | { kind: 'done'; id: number }
  | { kind: 'delete'; id: number }
  | { kind: 'list'; filter: 'all' | 'done' | 'todo' };

function run(cmd: Command) {
  switch (cmd.kind) {
    case 'add':    return addNote(cmd.text, cmd.tags); // TS знает: есть text и tags
    case 'done':   return markDone(cmd.id);
    case 'delete': return remove(cmd.id);
    case 'list':   return list(cmd.filter);
    default: {
      const never: never = cmd;  // если добавите вариант и забудете case — ошибка
компиляции
      throw new Error(`Unknown command: ${JSON.stringify(never)}`);
    }
  }
}
```

Это идеальная схема для задачи 1: **parse** (строки argv → `Command`) и **run** (`Command` → результат) разделены и тестируются отдельно.

5. Типизация ошибок

```
try { ... } catch (err) {        // err: unknown (в strict)
  if (err instanceof Error && 'code' in err && err.code === 'ENOENT') { ... }
}

// Свои классы ошибок — отличать «ожидаемые» ошибки от багов
class NotFoundError extends Error {
  constructor(public readonly id: number) {   // ← parameter property нельзя при type
stripping!
    super(`Заметка #${id} не найдена`);
  }
}
// Правильно для type stripping:
class NotFoundError extends Error {
  readonly id: number;
  constructor(id: number) {
    super(`Заметка #${id} не найдена`);
    this.name = 'NotFoundError';
    this.id = id;
  }
}
```

6. Полезные утилиты типов (повторение)

`Partial<T>`, `Required<T>`, `Pick<T, K>`, `Omit<T, K>`, `Record<K, V>`, `ReturnType<F>`, `Awaited<T>`, оператор `satisfies`.

```
type NewNote = Omit<Note, 'id' | 'createdAt' | 'updatedAt'>;  // DTO на создание
type NotePatch = Partial<Pick<Note, 'text' | 'done'>>;       // DTO на обновление
```

Упражнения дней 1–2 (`exercises/week2/`)

1. Настройте tsconfig из пункта 2, перенесите упражнения дня 2 недели 1 на TS с generics: `groupBy<T, K extends PropertyKey>(items: T[], fn: (x: T) => K): Record<K, T[]>`.
2. Напишите `parseNotesFile(raw: string): Note[]` — бросает понятную ошибку, если JSON невалиден или структура неверная. Без `any` и `as`.
3. Опишите `Command` для задачи 1 как discriminated union и функцию `parseCommand(argv: string[]): Command`.
4. `getEnvNumber(name: string, fallback: number): number` — читает переменную окружения, проверяет, что это число, иначе бросает ошибку.

День 3 Тесты на node:test

В Node встроены test runner и assert — для старта библиотеки не нужны (Vitest появится на 11-й неделе; API очень похож).

```
// test/notes.test.ts
import { describe, it, beforeEach } from 'node:test';
import assert from 'node:assert/strict';
import { addNote, markDone } from '../src/notes.ts';
import type { Note } from '../src/types.ts';

describe('addNote', () => {
  let notes: Note[];
  beforeEach(() => { notes = []; });

  it('добавляет заметку с id = 1 в пустой список', () => {
    const result = addNote(notes, 'Купить молоко');
    assert.equal(result.length, 1);
    assert.equal(result[0]?.id, 1);
    assert.equal(result[0]?.done, false);
  });

  it('не мутирует исходный массив', () => {
    addNote(notes, 'x');
    assert.deepEqual(notes, []);
  });

  it('бросает ошибку на пустой текст', () => {
    assert.throws(() => addNote(notes, '   '), /пуст/i);
  });
});

describe('markDone', () => {
  it('бросает NotFoundError для несуществующего id', () => {
    assert.throws(() => markDone([], 42), { name: 'NotFoundError' });
  });
});
```

```
node --test test/*.test.ts            # запустить
node --test --watch test/*.test.ts    # перезапуск при изменениях
```

assert	Для чего
`equal(a, b)`	строгое `===` для примитивов
`deepEqual(a, b)`	сравнение объектов/массивов по содержимому (в `/strict` — строгое)
`ok(value)`	truthy
`throws(fn, matcher)`	синхронная функция бросает ошибку
`await rejects(promise, matcher)`	промис отклоняется
`match(str, /re/)`	строка подходит под regex

Что тестировать в первую очередь

- **Чистую логику** (функции «данные → данные»): проще всего и ценнее всего.
- **Граничные случаи**: пустой список, несуществующий id, пустая строка, id после удаления.
- **Ошибки**: что бросается и с каким сообщением/типом.
- Код с файлами — через временную папку: `fs.mkdtemp(path.join(os.tmpdir(), 'notes-'))`, путь передать через `NOTES_FILE` или параметр.

> **Почему логику отделяют от ввода-вывода**
> Функция `addNote(notes, text)`, которая принимает массив и возвращает новый, тестируется в одну строку. Функция, которая сама читает файл, парсит argv и печатает в консоль, — почти не тестируется. Этот принцип («тонкий слой I/O, толстое чистое ядро») пригодится во всём бэкенде: контроллер → сервис → репозиторий.

Суббота Задача 1 на TypeScript

- Перенести задачу 1 на TS (strict, без `any`), разделить на `cli.ts` / `notes.ts` / `storage.ts`.
- Написать 6–8+ тестов на `notes.ts` и 1–2 теста на `storage.ts` через временную папку.
- Сделать 3–4 пункта из «Уровень+» на выбор (рекомендую: фильтры, теги, атомарная запись, `npm link`).

Воскресенье Streams

Теория ~1 ч, затем задача 2.

1. Зачем

`readFile` загружает файл в память целиком. Лог на 2 ГБ = 2+ ГБ памяти (или падение). Stream читает файл **кусками** (по 64 КБ), обрабатывает и забывает: память постоянная, сколько бы ни весил файл. Те же streams лежат под HTTP: тело запроса и ответа — это streams.

2. Четыре типа

Тип	Что делает	Примеры
Readable	источник данных	`fs.createReadStream`, `req` в HTTP-сервере, `process.stdin`
Writable	приёмник	`fs.createWriteStream`, `res`, `process.stdout`
Duplex	и то, и другое независимо	TCP-сокет
Transform	преобразует проходящие данные	`zlib.createGzip()`, шифрование, парсинг CSV

3. Чтение построчно — основной приём

```
import { createReadStream } from 'node:fs';
import { createInterface } from 'node:readline';

const rl = createInterface({
  input: createReadStream('access.log'),
  crlfDelay: Infinity,          // корректно для файлов с \r\n (Windows)
});

let count = 0;
for await (const line of rl) {  // async-итератор: строки по одной
  count++;
}
console.log(count);
```

4. pipeline — соединить потоки правильно

```
import { pipeline } from 'node:stream/promises';
import { createReadStream, createWriteStream } from 'node:fs';
import { createGzip } from 'node:zlib';

await pipeline(
  createReadStream('access.log'),
  createGzip(),
  createWriteStream('access.log.gz'),
);
```

`pipeline` корректно передаёт ошибки и закрывает все потоки при сбое. Старый `a.pipe(b)` этого не делает — избегайте.

5. Backpressure (обратное давление)

Если писать в поток быстрее, чем он успевает сбрасывать данные на диск/в сеть, данные копятся в памяти. `stream.write()` возвращает `false`, когда внутренний буфер переполнен, — значит, пора подождать события `'drain'`.

```
import { createWriteStream } from 'node:fs';
import { once } from 'node:events';

const out = createWriteStream('big.log');
for (let i = 0; i < 1_000_000; i++) {
  if (!out.write(`line ${i}\n`)) {
    await once(out, 'drain');     // буфер полон — ждём, пока освободится
  }
}
out.end();
await once(out, 'finish');
```

Без этой проверки генератор из задачи 2 будет есть сотни мегабайт памяти. Поэкспериментируйте: уберите проверку и сравните `process.memoryUsage().rss`.

6. Измерение памяти и времени

```
const start = performance.now();
// ... работа ...
const mb = (process.memoryUsage().rss / 1024 / 1024).toFixed(1);
console.error(`Время: ${(performance.now() - start).toFixed(0)} мс, память: ${mb} МБ`);
```

Проверь себя: неделя 2

1. Что делает type stripping? Почему при нём нельзя `enum` и parameter properties?
2. Зачем отдельно запускать `tsc --noEmit`, если Node и так запускает `.ts`?
3. Почему результат `JSON.parse` лучше присваивать в `unknown`? Что такое type guard?
4. Что такое discriminated union и зачем в `default` присваивание в `never`?
5. Чем `assert.equal` отличается от `assert.deepEqual`? Как протестировать async-ошибку?
6. Почему логику стоит отделять от ввода-вывода?
7. Что будет с памятью при `readFile` на файле 2 ГБ и при чтении через stream?
8. Назовите 4 типа streams с примерами.
9. Что такое backpressure и что означает `write()` → `false`?
10. Почему `pipeline` лучше `.pipe()`?

11. ТЗ задачи 1: CLI-менеджер заметок `notes`

Цель: консольная утилита для заметок с хранением в JSON-файле. Отрабатываем: аргументы командной строки, файлы, async/await, обработку ошибок, структуру проекта, тесты.

Ограничения

- Только встроенные модули Node (`typescript` и `@types/node` в devDependencies — можно). Аргументы — вручную или через `util.parseArgs`.
- ESM, асинхронные версии `fs`.

Команды (базовый уровень)

```
notes add "Купить молоко"        → Добавлена заметка #1
notes list                       → список всех заметок
notes done 1                     → отметить #1 выполненной
notes undone 1                   → снять отметку
notes delete 1                   → удалить #1
notes edit 2 "Новый текст"       → изменить текст
notes help                       → справка
```

Формат `list`:

```
#1 [ ] Купить молоко            (11.11.2026 14:30)
#2 [x] Позвонить маме           (10.11.2026 09:12)

Всего: 2, выполнено: 1
```

Пустой список — сообщение «Заметок пока нет. Добавьте: notes add "текст"».

Модель данных

```
interface Note {
  id: number;          // автоинкремент; id не переиспользуются после удаления
  text: string;
  done: boolean;
  createdAt: string;   // ISO
  updatedAt: string;   // ISO
}

interface NotesFile {
  nextId: number;      // подсказка: так проще не переиспользовать id
  notes: Note[];
}
```

Путь к файлу: переменная окружения `NOTES_FILE`, иначе `~/.notes/notes.json` (`os.homedir()`). Папку создавать при необходимости.

Обработка ошибок

Ситуация	Поведение
Файла нет	считать, что заметок нет; создать при первой записи
Невалидный JSON или неверная структура	понятное сообщение с путём к файлу; **файл не перезаписывать**; код 1
Нет заметки с таким id	`Заметка #7 не найдена`, код 1
id не целое положительное число	ошибка + подсказка по использованию команды, код 1
Пустой текст (или из пробелов) в `add` / `edit`	ошибка, код 1
Неизвестная команда / нет команды	справка, код 1 (для `help` — код 0)
Нет прав на запись	понятное сообщение, код 1

Ошибки — в `stderr` (`console.error`), результат — в `stdout`. Код выхода через `process.exitCode`. Пользователь **никогда** не должен видеть stack trace на ожидаемых ошибках.

Структура

```
notes-cli/
  src/
    cli.ts       — точка входа: parseCommand → run → вывод; ловит ошибки, ставит exitCode
    commands.ts  — parseCommand(argv): Command (discriminated union)
    notes.ts     — чистая логика: addNote, markDone, removeNote, editNote, filter...
    storage.ts   — load(path): NotesFile, save(path, data)
    format.ts    — форматирование вывода
    errors.ts    — NotFoundError, ValidationError, StorageError
  test/
    notes.test.ts
    commands.test.ts
    storage.test.ts
  package.json, tsconfig.json, README.md
```

Уровень+

1. **Фильтры:** `notes list --done`, `notes list --todo`.
2. **Поиск:** `notes search молоко` — без учёта регистра.
3. **Теги:** `notes add "Купить молоко" --tag shop --tag home`; `notes list --tag shop`; `notes tags` — все теги с количеством.
4. **JSON-вывод:** `notes list --json` (для скриптов: `notes list --json | jq`).
5. **Атомарная запись:** писать во временный файл рядом, затем `rename`. Вопрос себе: что будет со старыми данными, если процесс упадёт посреди обычного `writeFile`?
6. **Бэкап:** перед записью копировать старый файл в `notes.json.bak`.
7. **Глобальная команда:** `"bin": { "notes": "./src/cli.ts" }`, первая строка `#!/usr/bin/env node`, затем `npm link`.
8. **Цвета:** `util.styleText('green', 'текст')` — без chalk. Отключать, если вывод не в терминал (`process.stdout.isTTY`) или задана переменная `NO_COLOR`.
9. ***Сортировка:** `notes list --sort created|text --desc`.

Критерии готовности

- [ ] Все базовые команды и вся таблица ошибок.
- [ ] TypeScript strict, `tsc --noEmit` без ошибок, ни одного `any` и `as` для данных из файла.
- [ ] 8+ тестов, `npm test` зелёный.
- [ ] README: установка, примеры команд, где хранятся данные.
- [ ] Код на GitHub, осмысленные коммиты.

Подсказки

- `fs.readFile` на несуществующем файле бросает ошибку с `code === 'ENOENT'` — обрабатывайте именно этот код.
- `Number('12abc')` → `NaN`, `Number('')` → `0` (!). Для проверки id: `Number.isInteger(n) && n > 0`.
- Во время ручной проверки используйте отдельный файл: `NOTES_FILE=./tmp.json node src/cli.ts list`.
- Если застряли больше чем на 30 минут — приходите с конкретным вопросом и кодом.

12. ТЗ задачи 2: анализатор логов

Цель: обработать большой лог-файл с постоянным расходом памяти. Отрабатываем: streams, readline, backpressure, Map, производительность.

Часть А: генератор

`generate.ts --lines 1000000 --out access.log` — создаёт лог в формате:

```
2026-11-11T10:00:00.123Z 192.168.1.15 GET /api/users 200 123ms
2026-11-11T10:00:00.456Z 10.0.0.7 POST /api/orders 201 87ms
2026-11-11T10:00:01.001Z 192.168.1.15 GET /api/users/42 404 12ms
```

- ~50 разных IP (некоторые встречаются гораздо чаще — сделайте неравномерное распределение), ~15 путей, методы GET/POST/PUT/DELETE, статусы с реалистичными долями (большинство 2xx, немного 4xx, редкие 5xx), время 5–2000 мс, время по возрастанию.
- ~0.1% строк — «битые» (обрезанные или мусор) — анализатор должен их пережить.
- Запись через `createWriteStream` **с учётом backpressure**. Выведите время и память в конце.

Часть Б: анализатор

`analyze.ts access.log [--top 5] [--json]`

```
Файл: access.log (98.4 МБ), строк: 1 000 000, битых: 1 012

Статусы:      2xx: 91.2%   3xx: 2.1%   4xx: 5.9%   5xx: 0.8%
Время ответа: среднее 214 мс, медиана 120 мс, p95 980 мс, макс 1999 мс

Топ-5 IP:
  192.168.1.15   84 211
  10.0.0.7       61 002
  ...
Топ-5 путей:
  GET /api/users   220 140
  ...
Самые медленные пути (среднее время):
  POST /api/reports   1 450 мс
  ...

Обработано за 1.8 с, память 48 МБ
```

- Чтение **только** через stream + `readline`.
- Счётчики — `Map`. Битые строки считать, но не падать.
- Медиана и p95: подумайте, как их посчитать без хранения миллиона чисел. Подсказка: время ответа — целое от 0 до 2000; массив-гистограмма из 2001 счётчика.
- Время и память выводить в `stderr`, отчёт — в `stdout`.
- Ошибки: файла нет, это папка, нет прав — понятные сообщения и код 1.

Эксперимент (обязательно)

Сделайте вторую версию анализатора через `readFile` + `split('\n')`. Сгенерируйте файлы на 1 млн и 10 млн строк. Сравните время и память обеих версий, запишите результаты в README. Попробуйте ограничить память: `node --max-old-space-size=256 analyze-naive.ts big.log`.

Уровень+

1. `--from` / `--to` — фильтр по времени.
2. Поддержка `.gz`: если файл заканчивается на `.gz`, вставить `zlib.createGunzip()` в цепочку.
3. Чтение из `stdin`, если файл не указан: `cat access.log | node analyze.ts`.
4. Прогресс в `stderr`: «обработано 45%» (по количеству прочитанных байт относительно размера файла).
5. Тесты на функцию парсинга строки и на подсчёт перцентилей.

13. Ответы и источники

Упражнение «угадай порядок»

Запуск	Вывод
`order.cjs` (CommonJS)	`1, 6, 5, 4, затем 2 и 3 в любом порядке`
`order.mjs` (ESM)	`1, 6, 4, 5, затем 2 и 3 в любом порядке`
`io.mjs`	`всегда immediate, затем timeout`

Почему:

- **1, 6** — синхронный код выполняется первым, до любых очередей.
- **5 и 4** — микрозадачи, выполняются сразу после текущего синхронного кода. В CommonJS сначала очередь `nextTick`, потом промисы. В ESM модуль сам выполняется как асинхронная операция, и после его кода сначала дренируется очередь промисов — поэтому **4 раньше 5**.
- **2 и 3 на верхнем уровне** — порядок недетерминирован: `setTimeout(0)` на деле означает ≥1 мс. Если к моменту входа в фазу timers 1 мс уже прошла — сначала timeout, иначе цикл проходит дальше, и первым срабатывает immediate (фаза check).
- **Внутри I/O-колбэка** мы находимся в фазе poll; следующая фаза — check, поэтому `setImmediate` **всегда** раньше, чем timers следующей итерации.

Практический вывод: не стройте логику на порядке nextTick/промисов/таймеров. Если важен порядок — используйте `await`.

Упражнение block.js (суббота)

Тики останавливаются на 2 секунды, а после окончания цикла **не** «догоняют» пропущенные — один колбэк срабатывает сразу, дальше снова раз в 100 мс. Это и есть блокировка event loop: пока выполняется синхронный код, никакие колбэки (и никакие HTTP-запросы на сервере) не обрабатываются.

Источники

- **nodejs.org/learn** — официальные гайды; особенно «Asynchronous Work» и «The Node.js Event Loop».
- **nodejs.org/api** — документация модулей: `fs`, `process`, `stream`, `test`, `util`.
- **javascript.info** — разделы «Промисы, async/await», «Map и Set».
- **typescriptlang.org/docs/handbook** — «Narrowing», «Generics»; страница про `tsconfig` (Modules: nodenext).
- Доклад Philip Roberts — «What the heck is the event loop anyway?» (JSConf EU 2014).
- **roadmap.sh/nodejs** и **roadmap.sh/backend** — общая карта.

Дальше
Неделя 3 — HTTP с нуля на `node:http`. В конце недели 2 — срез знаний по вопросам «Проверь себя» и ревью кода задач 1–2.